

O que vimos até aqui

Transcrição

Paulo: No intervalo, eu mexi em um arquivo do nosso sistema. Porque eu quero que você dê o *pull* e veja as modificações. Inclusive, tem várias formas de darmos *pull* ou *push*. O Guilherme fez das maneiras mais explícitas.

Quando nossos arquivos de configuração estão bem configurados, para o `git pull` basta fazer `git pull`, não precisa colocar aquela URL gigante. Então, se fizer um `git pull` agora, ele vai informar que o `app.js` tem três linhas a mais. Se você abrir o `app.js` agora vai ver que tem uma modificação lá.



Guilherme: Vamos ver na web também como essa atualização aparece.

Paulo: E se você clicar no commit dá para ver o histórico de edições.

Guilherme: Sim. Antes tínhamos duas linhas, agora temos três linhas adicionadas aqui, assim como ele nos informou no terminal.

Paulo: Ele mostra essa tela de como era antes e como está depois da edição, isso é o que chamamos de "diff". É muito comum falarmos: "Me mostra o diff do seu commit, deixa eu ver o que você modificou". Esse é um jargão muito comum. Você também pode fazer o diff dessa versão com versões mais antigas. Dá para fazer diff tanto via web quanto na linha de comando.

Tem coisas que, para ser sincero, vai ser até mais fácil em linha de comando. E é muito comum usar a linha de comando, é raro um time de desenvolvimento que vá usar apenas a versão web. Esperamos que no futuro seja apenas web, mas hoje você precisa encarar a linha de comando no terminal e praticar bastante.

Guilherme: Como você falou, nesse repositório específico posso fazer só um `git pull`, sem passar a URL, o mesmo aconteceria para um `git push` também. Não preciso passar a origem principal.

Vou colocar aqui no `app.js` mais um `console.log()`: `console.log("Enviando para o GitHub")`. Salvei esse arquivo, vou para o terminal e fazer um `git commit app.js`. Aqui tem um ponto interessante, se eu faço modificações em mais de um arquivo, posso colocar só o ponto: `git commit .`, isso significa que quero pegar todo o meu projeto. Vou colocar uma mensagem: `git commit . -m "adicionando um console log"`. Vou dar um "Enter" e fazer o `git push` com esses dois, pressionar "Enter" e ele vai enviar para o repositório origem.

Quando voltarmos para o nosso repositório no site do GitHub já vai estar atualizado. Agora o arquivo `app.js` tem as edições do Paulo e a minha. Eu fiz o *pull*, fiz o commit e já fiz o *push* para enviar o código que eu fiz.

Lembrando, estou trabalhando sem nenhuma ramificação, um nome meio técnico que o pessoal dá para as divisões, existem várias estratégias para dividir o Git, para separar o que é código de teste, código de desenvolvimento e código principal. Quando trabalhamos em uma só raiz, que é o nosso código *main*, não precisamos especificar exatamente onde estamos trabalhando.

O que eu quero fazer agora é voltar a minha aplicação para um estado anterior. Na página do nosso repositório no GitHub, temos o ícone de um relógio que está com o número 8 do lado dele. Isso significa que temos 8 versões de história, 8 commits feitos.

Paulo: Não é quantos *push*, é quantos commit, porque às vezes num *push* tem mais de um commit. Ainda não fizemos isso, mas poderia acontecer. Clicando aí neste ícone do histórico dá para listarmos todos e ver a história inteira da aplicação. Todas as modificações.

Guilherme: Eu quero fazer algo interessante, eu quero voltar para a sua modificação. Não que a modificação que eu fiz esteja ruim, mas eu quero retornar para o estado do projeto que estava naquele momentodo seu commit.

Paulo: Claro que daria para clicar em um desses, ver como era o arquivo antes, copiar e colar, dar um commit e um *push*. Você passar por cima. Mas o Git tem mecanismos para você falar: "Por favor, volte para como era na época de determinado commit".

Trabalhar com Git e GitHub não é só para fazer o mesmo que um Dropbox, por exemplo, é muito mais. Ele guarda todas as versões, todos os históricos, você pode ir, pode voltar, ver quem fez, qual o tamanho, se isso estava conectado com aquilo. É realmente feito para trabalhar com muito código. E é muito frequente vermos o histórico dos arquivos, porque se você vê como era antes, começa a entender as modificações que foram feitas. Precisamos ganhar esse hábito, não tenham medo de entrar no GitHub e ver esses históricos.

Guilherme: No GitHub local via linha de comando também temos acesso a essas modificações e vou deixar até um "Para saber mais" explicando como conseguimos visualizar o log, saber a hora, de quem é a autoria daquele commit, etc.

Eu quero visualizar essas mensagens do histórico e voltar o meu projeto para o estado desse commit que o Paulo fez. Então, na linha de comando, para visualizar o log das

modificações vou colocar: `git log --oneline` . Eu quero voltar para o commit "a0c893b console log indicando modificações do Paulo". Então, vou copiar essa sequência de caracteres "a0c893b" que identifica esse commit como sendo único e vou colocar o seguinte comando: `git restore` .

O `git restore` vai fazer a gente voltar para um determinado momento da aplicação. Mas preciso indicar para ele qual hash vamos utilizar. Para indicar qual hash vamos utilizar vou colocar `--source` e vou passar o hash com aqueles caracteres que copieei do commit que o Paulo fez:

```
git restore --source a0c893b
```

[COPIAR CÓDIGO](#)

Mas ainda tem uma coisa interessante, se eu escrever, por exemplo, `app.js` , vou pegar esse commit que o Paulo fez naquele momento e vou deixar todo o `app.js` naquele estado. Mas se o Paulo tivesse feito modificações em diversas partes da minha aplicação

e quero voltar a minha aplicação para aquele determinado momento, eu coloco só o ponto(.), esse é o comando que vamos utilizar, vamos inserir o ponto:

```
git restore --source a0c893b .
```

[COPIAR CÓDIGO](#)

Agora sim vou pressionar "Enter" e não recebi nenhuma mensagem diferente. Se eu der um `git log --oneline`, olha que interessante. Ele vai continuar da mesma forma.

Paulo: Ele restaurou, mas só localmente, ainda não commitamos as mudanças. Agora podemos falar que vamos commitar o arquivo tal, voltando atrás de acordo com determinado commit.

```
git commit app.js -m "voltando para o estado do paulo"
```

[COPIAR CÓDIGO](#)

Agora o git status vai falar que só falta o *push*. Vamos dar um `git status` para vermos como está. Está tudo commitado, não tem nada sujo. Nosso repositório local está preparado, mas ele está na frente do original por 1 commit, então você tem que empurrar esse commit para lá. Ou dá até para cancelar o commit se tiver feito algo errado, tem vários comandos, muitas opções para trabalhar com os commits que fizemos. Agora nós queremos empurrar, queremos fazer o *push*.

Guilherme: Podemos ver no navegador que as modificações que eu havia feito não saíram porque eu voltei ao estado anterior, mas não havia feito um commit. Agora sim fiz o commit, e vou fazer o comando `git push` .

Agora no navegador, quando atualizamos a página, aquela linha que eu havia feito sumiu, voltamos para o commit que o Paulo havia criado.